Unit II – Inter-Process Communication (IPC)

Types of Processes

A process can be of two types:

1. Independent Process

An **independent process** executes without being affected by other processes. Its execution does not depend on or influence the execution of another process.

2. Cooperating Process

A **cooperating process** can **affect or be affected by other processes**. Such processes may need to share information or resources.

Cooperation between processes is useful because it can improve:

- **Computational speed** – multiple processes can work together.
- **Convenience** – processes can share information and resources.
- **Modularity** – a system can be divided into smaller cooperating processes.

Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism that allows processes to:

- **Communicate with each other**
- **Exchange information**
- **Synchronize their actions**

In simple words, IPC enables cooperating processes to work together by sharing data or sending information.

There are **two main methods of IPC**:

1. **Shared Memory**
2. **Message Passing**

An operating system can support both methods.

---

1. Shared Memory

In the **shared memory method**, two or more processes are given access to a **common memory area**.

The processes can use this common memory to exchange information.

Working

Suppose **Process 1** and **Process 2** are executing at the same time:

1. Process 1 performs some computation or obtains some information.
2. It stores that information in the **shared memory**.
3. When Process 2 needs the information, it reads it from the shared memory.
4. Process 2 then uses the information and acts accordingly.

Thus, shared memory can be used both for:

- **Extracting information** stored by another process.
- **Delivering information** to another process.

Simple Idea

Process 1 ⟶ Shared Memory ⟷ Process 2

(Common Memory)

Important Point

In shared memory communication, the processes share a common memory area or variables, but **the programmer is responsible for deciding how the processes coordinate and use that shared memory**.

Example

A common example of shared memory communication is the **Producer–Consumer Problem**, where the Producer and Consumer share a common memory area called a **buffer**.

i) Shared Memory Method – Producer–Consumer Problem

The **Producer–Consumer problem** is an example of IPC using **shared memory**.

There are two processes:

- **Producer** – produces items.
- **Consumer** – consumes the produced items.

Both processes share a common memory area called a **buffer**.

Working

1. The Producer produces an item and places it in the buffer.

2. The Consumer takes an item from the buffer and consumes it.

Types of Buffer

1. Unbounded Buffer

- The buffer has **no size limit**.
- The Producer can continue producing items without waiting.

2. Bounded Buffer

- The buffer has a **fixed size**.
- The Producer can produce items only until the buffer becomes full.
- If the buffer is full, the **Producer waits** until the Consumer consumes an item.
- If the buffer is empty, the **Consumer waits** until the Producer produces an item.
- If items are available, the Consumer consumes them.

Simple Flow

Producer → produces item → Shared Buffer → Consumer

In short: The Producer and Consumer communicate through a shared buffer and must coordinate their actions to avoid producing into a full buffer or consuming from an empty buffer.

---

ii) Message Passing Method

In the **message passing method**, processes communicate by **sending and receiving messages** without sharing a common memory area.

Suppose two processes, **P1 and P2**, want to communicate.

Steps

1. **Establish a communication link** between the processes.
    - If a link already exists, it can be reused.
2. **Exchange messages** using communication operations called **primitives**.

The two basic primitives are:

- send(message, destination) – sends a message to another process.
- receive(message, source) – receives a message from another process.

The destination or source may sometimes be omitted:

- send(message)
- receive(message)

Message Size

Messages can be of two types:

Fixed-Size Message

- Every message has the same size.
- Easier for the **operating system designer**.
- Less convenient for the **programmer**.

Variable-Size Message

- Messages can have different sizes.
- More convenient for the **programmer**.
- More complex for the **operating system designer**.

Structure of a Message

A standard message generally has two parts:

1. Header

Contains control and identification information such as:

- Message type
- Destination ID
- Source ID
- Message length
- Sequence number
- Priority
- Buffer/control information

2. Body

Contains the **actual data or information** being sent.

Message

├— Header → Control and identification information

└— Body → Actual data

FIFO Message Order

Messages are generally transmitted using **FIFO (First In, First Out)** order.

This means the message that is sent first is normally received first.

Quick Difference

Shared Memory	**Message Passing**
Processes share a common memory area.	Processes do not share memory.
Data is exchanged through shared memory.	Data is exchanged through messages.
Example: Producer–Consumer with a shared buffer.	Uses send() and receive() operations.

Message Passing through Communication Link

In **message passing**, processes communicate through a **communication link**. A communication link is the connection or channel through which messages are sent between processes.

The communication link can be implemented in two ways:

1. **Direct Communication**
2. **Indirect Communication**

Questions to Consider When Implementing a Communication Link

While designing a communication link, the following questions are important:

1. **How is the communication link established?**
2. **Can more than two processes share the same link?**
3. **How many links can exist between a pair of communicating processes?**
4. **What is the capacity of the link?**
5. **Can the link be unidirectional or bidirectional?**

Link Direction

- **Unidirectional:** Communication occurs in only one direction.
- **Bidirectional:** Communication occurs in both directions.

Capacity of a Communication Link

Every communication link has a **queue** that temporarily stores messages. The capacity of the link determines how many messages can be stored in this queue.

There are three types:

1. Zero Capacity

The link cannot store any waiting messages.

- The sender must wait until the receiver receives the message.
- The sender and receiver must synchronize directly.

2. Bounded Capacity

The queue has a **fixed maximum size**.

- A limited number of messages can be stored.
- If the queue becomes full, the sender must wait.

3. Unbounded Capacity

The queue has **no fixed limit** on the number of messages it can hold.

- The sender can continue sending messages without waiting for the queue to become full.

In **non-zero capacity links**, after sending a message, the sender may not immediately know whether the receiver has actually received it. If confirmation is required, the sender and receiver must communicate explicitly.

---

1. Direct Communication Link

In **direct communication**, processes communicate by using the **specific process identifier (ID)** of the other process.

The sender directly identifies the receiver while sending the message.

Process P1 ——————————► Process P2

Direct Communication

Example: send(P2, message) means that a process sends a message directly to Process P2.

A limitation is that the communicating process must know the identity of the other process. In some situations, such as a **print server**, it may be difficult to identify the sender in advance.

---

2. Indirect Communication Link

In **indirect communication**, processes do not communicate directly with each other. Instead, they communicate through a **shared mailbox or port**.

A mailbox contains a **queue of messages**.

Working

1. The sender places a message in the mailbox.
2. The message remains in the mailbox queue.
3. The receiver picks up the message from the mailbox.

Sender ——► Mailbox / Port ——► Receiver

(Message Queue)

In short:

- **Direct communication:** Processes communicate using each other's **process IDs**.
- **Indirect communication:** Processes communicate through a **shared mailbox or port**.

Message Passing Through Exchanging Messages

IPC can take place between processes running on the **same computer** or on **different computers in a network/distributed system**.

During message passing, a process may either **wait (block)** or **continue execution (non-block)** while sending or receiving a message.

A **blocked process** is a process that is waiting for an event, such as a message becoming available, a resource becoming available, or an I/O operation completing.

There are two types of message passing:

1. **Synchronous (Blocking) Message Passing**

2. **Asynchronous (Non-Blocking) Message Passing**

---

1. Synchronous or Blocking Message Passing

In **blocking communication**, a process waits until the required operation is completed.

Blocking Send

The sender sends a message and then **waits until the message is received** by the receiver.

Blocking Receive

The receiver waits until a **message becomes available**.

Therefore, blocking communication is called **synchronous communication** because the sender and receiver must synchronize their actions.

---

2. Asynchronous or Non-Blocking Message Passing

In **non-blocking communication**, a process does not have to wait.

Non-Blocking Send

The sender sends the message and **immediately continues its execution**.

Non-Blocking Receive

The receiver checks for a message:

- If a valid message is available, it receives it.
- If no message is available, it receives a **null value** and continues execution.

Therefore, non-blocking communication is called **asynchronous communication**.

Preferred Combinations

There are three commonly used combinations:

1. Blocking Send + Blocking Receive

- Sender waits until the message is received.
- Receiver waits until a message is available.

Both processes may wait for each other.

---

2. Non-Blocking Send + Non-Blocking Receive

- Sender sends the message and continues.
- Receiver checks for a message and continues even if no message is available.

Neither process is forced to wait.

---

3. Non-Blocking Send + Blocking Receive — Mostly Used

- Sender sends the message and continues its execution.
- Receiver waits until a message becomes available.

This is commonly preferred because:

- The sender may need to send messages to other processes and can continue its work.
- The receiver often needs the received information before continuing its execution.

Quick Comparison

Type	Sender	Receiver
Blocking Send + Blocking Receive	Waits	Waits
Non-Blocking Send + Non-Blocking Receive	Continues	Continues
Non-Blocking Send + Blocking Receive	Continues	Waits for message

Easy memory point:
Blocking = Wait = Synchronous
Non-blocking = Continue = Asynchronous

Indirect Message Passing

In **indirect message passing**, processes communicate through a **mailbox**, also called a **port**, instead of communicating directly with each other.

Each mailbox has a **unique ID**, and processes can communicate only when they **share a common mailbox**.

Sender ——► Mailbox / Port ——► Receiver

(Message Queue)

Main Features

- A communication link is established only when processes share a **common mailbox**.
- A single mailbox can be shared by **multiple processes**.
- A pair of processes can share **multiple communication links/mailboxes**.
- Communication can be **unidirectional or bidirectional**.

Basic Operations

The following operations are performed:

1. **Create** a mailbox.
2. Use the mailbox to **send and receive messages**.
3. **Destroy** the mailbox when it is no longer needed.

Message Passing Primitives

- send(A, message) → Sends a message to mailbox **A**.
- receive(A, message) → Receives a message from mailbox **A**.

Problem with Multiple Receivers

Suppose several processes share the same mailbox.

If **P1 sends a message** to the mailbox, the question is:

Which process should receive the message?

This problem can be solved in three ways:

1. Allow only **two processes** to share a mailbox.
2. Allow only **one process at a time** to execute the receive() operation.
3. Allow the system to **select a receiver** and notify the sender.

Private and Shared Mailboxes

A mailbox can be:

Private Mailbox

Used by a single **sender–receiver pair**.

Shared Mailbox

Used by **multiple sender and receiver processes**.

Port

A **port** is an implementation of a mailbox.

It can support:

- **Multiple senders**
- **One receiver**

Ports are commonly used in **client–server applications**, where:

- Clients → send messages.
- Server → receives messages.

The port is usually:

- Owned by the receiving process.
- Created by the operating system at the request of the receiver.
- Destroyed by the receiver or when the receiving process terminates.

Mutual Exclusion in a Mailbox

If multiple processes try to receive messages from the same mailbox, **mutual exclusion** can be used.

A **mutex mailbox** is shared by multiple processes.

- The sender can send a message without waiting (**non-blocking**).
- The first process that executes receive() enters the **critical section**.
- Other processes are blocked and must wait.

This ensures that only **one process receives a message at a time**.

Producer–Consumer Using Message Passing

The Producer–Consumer problem can also be implemented using **message passing**.

- The **Producer** places produced items inside messages.
- These messages are sent to a **mailbox**.
- The **Consumer** receives messages from the mailbox.
- The Consumer can consume an item when **at least one message is available**.

Producer —► Message / Mailbox —► Consumer

Examples of IPC Systems

System	IPC Method
POSIX	Shared Memory
Mach	Message Passing
Windows XP	Message passing using Local Procedure Calls

IPC in Client–Server Architecture

Common communication mechanisms used in client–server architecture are:

1. **Pipe**
2. **Socket**
3. **Remote Procedure Call (RPC)**

These mechanisms allow clients and servers to communicate and exchange information.

Quick Memory Summary

Indirect IPC → Mailbox/Port → send() and receive() → Multiple processes can share → Mutual exclusion handles multiple receivers.